**ISMS & Compliance ✅✅✅**
ISMS-Grundlagen und ISO/IEC 27001 verstehen
Security-Policies, Governance und Compliance-Anforderungen einhalten

**Jira & Projektmanagement ✅**
Jira-Workflows und Projektstrukturen pflegen
Projektphasen, Meilensteine und Abhängigkeiten verwalten
Status, Fortschritt, Reporting und Projekt-Governance unterstützen

**Copilot Studio & Agentic AI**
Shared Agents für Cloud Service Security Check und Secure Software Acquisition entwickeln und pflegen ✅
Policies, Formulare und erforderliche Artefakte integrieren
Jira per API anbinden
Human-Approval- und Validierungsschritte sicherstellen
Agentic-AI-Funktionen implementieren
kontrollierte, auditierbare Kommunikation statt vollautomatischer E-Mails umsetzen

**Sandbox & Runtime**
Geeignete Sandbox-Lösung mit AppSec und SOC evaluieren und auswählen
bei Bedarf Runtime-Umgebungen aufsetzen
Sandbox in den Secure Software Acquisition Process integrieren
Sandbox in Copilot Studio integrieren

**Governance & Dokumentation ✅✅**
automatisierte und teilautomatisierte Prozesse dokumentieren
Agents, Tools, Skripte, Konfigurationen und Prozessdefinitionen versionieren
Change-Management-Prozesse einhalten
Auditability, Traceability und Transparenz sicherstellen

## Threat Modeling:

Threat Modeling = Welche Angriffswege entstehen durch Architektur/Design?

Threat Modeling mit STRIDE: Du gehst Komponenten und Datenflüsse durch und fragst nach sechs Kategorien:

| | |
|---|---|
| **S**poofing | Identität vortäuschen? |
| **T**ampering | Daten manipulieren? |
| **R**epudiation | Aktion abstreiten? |
| Information Disclosure | Datenleak? |
| **D**enial of Service | Dienst lahmlegen? |
| **E**levation of Privilege | mehr Rechte erhalten? |

Bsp API: Mobile App → API Gateway → Backend → Database
**Threat**: User kann fremde Rechnungen abrufen.
**Attack**: IDOR: GET /invoice/1 → 2 → 3
**Mitigation**: Backend muss Ownership prüfen.
**Security Requirement**: User darf ausschließlich Rechnungen abrufen, deren owner_id seiner eigenen user_id entspricht.

## Git:

git init → Neue Repository init
git clone [URL] → Repository lokal herunterladen
git add [Datei] → Änderungen zum Commit vorbereiten
git commit -m "Hi" → speichern mit Nachricht “Hi”
git push → Änderungen hochladen
git pull → Änderungen herunterladen & mergen mit lokal
git branch [name] → Neue branch erstellen
git merge [branch] →
git status → Status anzeigen
git log → Commit-Historie anzeigen

**Pull Request** = Review-/Merge-Anfrage

**Commit** = Ein einzelner gespeicherter Änderungspunkt. Commit abc123

**Version** = Kennzeichnung eines bestimmten Softwarezustands. v2.4.0 = MAJOR.MINOR.PATCH

**Release** = Eine bewusst veröffentlichte/bereitgestellte Version inklusive ggf.: Release Notes, Artefakte, Approval, Deployment

**Copilot Studio**: = platform to build, manage and publish AI agents and workflows.
**Agent** = AI system that uses instructions, knowledge and tools to answer or perform actions.

**Core Components**:
Model → reasoning / language engine
**Harness / Runtime** → Everything around the model that helps the agent work: knowledge, tools, skills, memory, etc.
Instructions → role, behavior
Knowledge → information used to answer
Tools → external actions / APIs / workflows
Skills → reusable capability/instructions
Flows → deterministic multi-step automation

**Skill** → **Define**: When to use it → What it should do → Required information → Rules/constraints → Expected result.

**Tools / Integration** → allow the agent to interact with external systems.
**Connector** → prebuilt integration
**API** → direct system communication
**MCP** → standardized way to expose tools/resources to agents
**Flow** → deterministic automation

**Agent Flow** Example: Request → Check data → Human approval → Create Jira ticket → Return result
Trigger
→ Actions
→ Conditions
→ optional Human Approval
→ Output
**Approval Gates** → Ein Kontrollpunkt, den ein Prozess erst verlassen darf, wenn eine definierte Genehmigungsbedingung erfüllt wurde.

**Testing Agent**
Functional → correct answer/action?
Grounding → correct trusted source?
Negative → handles missing/invalid input safely?
Adversarial → resists manipulation/prompt injection?
Authorization → only permitted actions/data?
Approval → human approval cannot be bypassed?

**Agent Security**:
Deterministic controls → enforce critical rules in code/workflows, not only prompts. Soft = Verhalten anweisen. Hard = Verhalten technisch erzwingen.
Least Privilege → tools/agent get only required permissions
Human Approval → required for sensitive actions
Untrusted Content → documents, websites and user input are data, not trusted instructions
Prompt Injection → especially indirect injection from external content like PDF.
Authorization → validate user permissions before accessing data or performing actions.

**Publish** → creates a live version of the agent

Channels → make the published agent available to users (e.g. Teams, Microsoft 365 Copilot, web/apps)

Important Copilot Studio Flow:
User Request → Agent → Instructions → Knowledge → Tool/Flow → Human Approval → Jira/API → Result

Best Model Selection practice:
**Default/stable model** → Prefer for production.
**Preview / Experimental model** → Don’t choose; availability and support can change.
**More reasoning** → Can improve complex tasks, but usually increases latency and cost.

Knowledge Best practice → Use trusted / official sources.
**Upload / Sync source** → Better for documents that change regularly.
**Disable Web-Search**.
Settings => Knowledge => **Disable Use general knowledge**.
Veränderbare Sachen wie Policies nicht als Instructions, sondern als Knowledge angeben.

- bsp: Instruction:
You are an internal IT Service Desk agent for [...].
Your role is to answer questions using only the IT Service Desk Knowledge Base.
* Only answer questions that are related to the IT Service Desk Knowledge Base.
* Base your answers only on information available in the Knowledge Base.
* Do not invent, assume, or provide information that is not supported by the Knowledge Base.
* If a user asks about a topic outside the scope of the IT Service Desk Knowledge Base, politely explain that you can only assist with IT Service Desk-related questions.
* If the Knowledge Base does not contain enough information to answer a question, clearly state that you do not have sufficient information rather than guessing.
* Keep answers clear, concise, and helpful.

- bsp: Skill:
When an employee requests time off, draft a leave request email addressed to leave@nordfalk.com.
Use the requirements defined in [source].
Use the example in [source] as a reference for the email structure and format.
If any required information is missing, ask the employee for it before drafting the email.
Do not guess, assume, or invent missing information.

**Copilot Studio mit Jira verbinden**:
Model: Copilot Agent → Action / Flow → Connector / HTTP → Jira REST API → Jira

Bsp: Prompt an den Agent: „Erstelle einen Software Security Review.“

Dann könnte ein Flow:

Agent-Input übernehmen
JSON erzeugen
Jira REST API aufrufen
Jira-Issue erstellen
Issue-Key erhalten
Issue-Key an Agent zurückgeben

Beispiel:

Agent
↓
CreateSecurityReview()
↓
POST /rest/api/3/issue
↓
Jira
↓
SEC-481

Dein Lernziel: REST-Grundlagen → JSON → Authentication → Jira API → Power Automate/Copilot Connector.

**Container**:  = isolierte Verpackung für eine Anwendung + benötigte Runtime, Libraries und Konfiguration.

Beispiel:

Container
├── App
├── Runtime, z. B. Python
├── Dependencies
└── Konfiguration

Vorteil: Dieselbe Umgebung läuft konsistent in Dev → Test → Staging → Production.

Docker = verbreitete Container-Technologie.

Image = Vorlage/Paket
Container = laufende Instanz eines Images

Docker Image → Container startet → App läuft

**Jira**: → Tool to plan, organize and track work. **Workflow**: → To Do → In Progress → Review → Done

**Responsibilities**:
Assignee → Who works on the ticket?
Reporter → Who created/requested it?
Due Date / Priority → When and how important?

**Project Management**:
Task ist eine Work-Item-Art, nicht automatisch Unterebene.
Status = Zustand; Transition = erlaubter Wechsel.
Blocked dokumentieren: Nicht nur Status, sondern Blocker, Ursache, Owner, erwartete Lösung und Impact.
Board → Visual overview of ticket status.
Backlog → Planned or waiting work.
Epic → Tasks → Subtasks: Breaks large objectives into smaller pieces of work.
Dependency → One task depends on another.
Milestone → Important project checkpoint.

**Communication**
Comment → Updates, questions, decisions.
Attachment → Files or evidence.
History → Changes made to the ticket.

**Search**
Filter → Shows selected tickets.
**JQL** → Advanced Jira search. e.g.: assignee = currentUser() → Show my tickets.
project = APPSEC AND status != Done

```
project = APPSEC
AND status = Blocked
ORDER BY priority DESC
```

```
project = APPSEC
AND assignee = currentUser()
AND statusCategory != Done
```

**Dashboard** → Custom overview of project information and progress.

**Important Jira Flow**:
Project → Epic → Task → Assignee → Due Date/Priority → Dependencies → Status → Board → Reporting

**Reporting** → Are we on track, delayed, or blocked?

**nicht**: Agent integration – In Progress.

**Sondern beispielsweise**:

Copilot–Jira Integration
Status: Blocked
Blocker: Service account approval pending
Dependency: SEC-142
Target: 12 Sep
Risk: Test environment unavailable
Next action: IAM approval

Das ist Projekttransparenz.
Custom Fields: Use-Case-spezifische Metadaten wählen, z. B. Vendor, Business Owner, Data Classification, Risk Rating.

## Cloud Security Assessment:

**Identity & Access**
SSO?
MFA?
RBAC?
Least Privilege?
Admin Accounts?

**Data Protection**
Welche Daten?
Retention?
Deletion?
Encryption?
Backups?
Training?

**Infrastructure**
Hosting?
Tenant Isolation?
Network Security?

**Application Security**
Secure SDLC?
Vulnerability Management?
Penetration Tests?
Patch Management?

**Operations**
Logging?
Monitoring?
Incident Response?
Business Continuity?

**Third Parties**
Subprocessors?
Certifications?
Supplier Risk?

EU Hosting ≠ automatisch nur EU-Verarbeitung:
**Wo können Daten gespeichert, verarbeitet, unterstützt, gesichert und eingesehen werden?**

**Tenant Isolation** sorgt dafür, dass Company B niemals auf Rhenus-Daten zugreifen kann:

SaaS Platform
├── Rhenus Data
├── Company B Data
└── Company C Data

**Third-Party Risks**:
Data Breach beim Vendor
Service-Ausfall
Supply-Chain-Kompromittierung
unautorisierter Zugriff auf eure Daten
Datenverlust / falsche Löschung

# Secure Software Acquisition

**Quick Check**: Was soll eingesetzt werden? → Use Case → Daten → Zugriffe → Vendor → IAM → Retention/Löschung → Hosting/Processing → Subprocessors → Tenant/Cloud Security → Security Practices → AI → Evidence → Risk → Controls → Residual Risk → Recommendation → Human Decision

**Master Check**:

## A. Requester Intake

**Prüfen**:

Welche Software / welcher Vendor / welche Version soll eingesetzt werden?
Softwaretyp: SaaS, Desktop, Extension, Library, Developer Tool, Cloud Service, AI Tool?
Wer sind Requester, Business Owner und ggf. Technical Owner?
Was ist der konkrete Use Case und warum wird die Software benötigt?
Pilot oder Production, wie viele Nutzer, welche Teams?
Business Criticality und gewünschter Go-Live?
Welche Daten werden verarbeitet und welche Data Classification haben sie?
Sind PII, Source Code, Confidential Data, interne Dokumente oder Secrets betroffen?
Werden Dateien, Repositories oder andere Unternehmensdaten hochgeladen bzw. synchronisiert?
Auf welche internen Systeme / Cloud Services / Repositories soll die Software zugreifen?
Welche Rechte werden benötigt: Read, Write, Delete, Admin?
Werden API-Integrationen oder Service Identities benötigt?
(bei lokaler Software/Extension) welche lokalen bzw. Browser-Rechte werden benötigt?
(bei AI) welche Unternehmensdaten gehen an die AI?
(bei AI) werden Source Code, Dateien oder Repositories verarbeitet/indexiert?
(bei AI-Agent) welche Aktionen und externen Systeme darf der Agent verwenden?
Gibt es regulatorische, vertragliche oder interne Security-Anforderungen?
Gibt es bereits ein früheres Assessment oder bekannte Security Concerns?
Welche Informationen sind Unknown und wer muss sie liefern?

**Dokumentieren**

Produkt / Vendor / Version
Requester / Business Owner / Technical Owner
Use Case / Nutzer / Pilot oder Production
Business Criticality / Target Go-Live
Daten + Classification
PII / Source Code / Secrets / Confidential Data
Systeme / Integrationen
benötigte Permissions
API / Service Identity
AI-/Repository-Nutzung
besondere Anforderungen
offene Fragen
Question → Owner → Due Date
Next Action

## B. Security Assessment

**Prüfen**:

Welche Kundendaten erhält, verarbeitet und speichert der Vendor?
Welche Retention gilt für Produktionsdaten, Logs, Backups, temporäre Daten und ggf. AI-Indizes?
Können Daten auf Anfrage und nach Vertragsende vollständig gelöscht werden?
Können Daten exportiert und Retention-Zeiten reduziert werden?
Wo werden Daten gespeichert, verarbeitet, geloggt, gesichert und für Disaster Recovery vorgehalten?
Aus welchen Ländern können Vendor-/Support-Mitarbeiter auf Daten zugreifen?
Können Daten die gewünschte Region verlassen?

Welche Subprocessors / Cloud-/Logging-/Support-/AI-Provider werden eingesetzt, welche Daten erhalten sie und wo verarbeiten sie?
Sind SSO und MFA vorhanden und erzwingbar?
Unterstützt das Produkt RBAC / Least Privilege und getrennte Admin-Rollen?
Gibt es kontrollierte Service Accounts / API Identities sowie Provisioning und Deprovisioning?
Können lokale Accounts trotz SSO genutzt werden und ggf. deaktiviert werden?
Wie wird Vendor-/Support-Zugriff genehmigt, begrenzt und geloggt?
Sind Daten in Transit, at Rest und in Backups angemessen verschlüsselt?
Gibt es Key Management, ggf. DLP / Redaction und Schutz vor sensiblen Daten in Logs?
(bei SaaS) wie werden Tenants getrennt und Cross-Tenant-Zugriffe verhindert?
Gibt es Secure SDLC, Code Review, SAST, SCA, Secret/Dependency Scanning und Security Testing?
Sind DEV/TEST/PROD, Change Management und Release Controls sinnvoll getrennt?
Gibt es regelmäßige Vulnerability Scans, Patch-/CVE-Management und definierte Remediation-Zeiten?
Wann war der letzte relevante Pentest und gibt es offene Critical-/High-Findings?
Gibt es einen Vulnerability-Disclosure-Prozess?
Werden Security-, Login-, Admin-, Permission-, Datenzugriffs-, API- und Konfigurationsereignisse sinnvoll geloggt?
Können relevante Logs exportiert bzw. an ein SIEM angebunden werden?
Gibt es einen Incident-Response-Prozess mit Security Contact und definierter Notification-Frist?
Gibt es angemessene Backup-, Restore-, Disaster-Recovery-, RTO-/RPO- und Availability-Maßnahmen?
(bei Software/Packages) sind Third-Party Dependencies / SBOM / Open-Source-Komponenten nachvollziehbar?
(bei AI) werden Kundendaten für Training verwendet und kann dies deaktiviert werden?
(bei AI) wie lange werden Prompts, Responses und ggf. Repository-Indizes gespeichert?
(bei AI) welcher Model Provider verarbeitet welche Daten?
(bei AI) können sensible Dateien / Secrets von Indexing und Verarbeitung ausgeschlossen werden?
(bei AI) sind Prompt Injection und untrusted Content berücksichtigt?
(bei AI-Agent) welche Tools und Permissions besitzt der Agent, und sind sie auf das Notwendige begrenzt?
(bei AI-Agent) ist Human Approval für kritische Aktionen technisch geschützt?
(bei AI-Agent) werden Agent Actions und Agent-Version nachvollziehbar geloggt?
Gibt es ISO 27001, SOC 2, Pentest oder andere relevante Assurance Evidence?
Passt deren Scope und Aktualität zum konkreten Produkt / Use Case?
Welche Assets sind durch unseren Use Case betroffen?
Welche tatsächliche Exposure entsteht: Daten, Systeme, Rechte, Third Parties, Blast Radius?
Welche konkreten Threats / Weaknesses entstehen?
Welche Auswirkungen auf Confidentiality, Integrity und Availability sind realistisch?
Wie hoch sind Impact und Likelihood?
Welche Existing Controls reduzieren das Risiko bereits?
Welche zusätzlichen Controls sind erforderlich?
Kann das Risiko durch weniger Nutzer, Read-only, eingeschränkte Repositories, geringere Permissions, geringere Retention oder einen Pilot reduziert werden?
Welches Residual Risk bleibt nach diesen Controls?
Welche Unknown-Punkte könnten das Risk Rating noch verändern?
Ist Risk Acceptance erforderlich und wer ist Risk Owner?
Welche Recommendation passt: Approve / Approve with Conditions / Request More Information / Investigate / Reject / Risk Acceptance required?
Welche Conditions gelten konkret?
Ist eine Pilot-, Zeit- oder Scope-Begrenzung notwendig?
Wer darf die finale Human Decision treffen?
Ist Human Decision von AI-/AppSec-Recommendation getrennt?
Müssen Expiry Date oder Reassessment Date gesetzt werden?

**Evidence**: Nur relevante Nachweise sammeln, z. B.:

ISO 27001 + Scope
SOC 2
Pentest / Retest
Vendor Security Questionnaire
Security Whitepaper
Subprocessor List

Data Residency / Retention Documentation
relevante AI-/Model-Provider-Dokumentation
technische Tests / Screenshots
Dokumentieren
relevante Vendor-Antworten
Daten / Retention / Deletion / Backups
Hosting / Processing / Support Locations
Subprocessors
IAM / Support Access
Encryption / Tenant Isolation
Secure Development / Vulnerability Management
Logging / Incident / Availability
AI-spezifische Ergebnisse
relevante Evidence
Assets / Exposure
Findings / Weaknesses
konkrete Risk Scenarios
Impact / Likelihood / Risk Rating
Existing Controls
Required Controls
Residual Risk
Recommendation
Conditions
Risk Owner
Human Approver
Final Decision
Begründung
offene Unknowns
Question / Condition → Owner → Due Date
Review / Expiry Date
Next Action

**2. Code ändern**.

**Vor Commit**:

Secret Scan
neue Dependencies prüfen
neue Permissions prüfen
neue externe Verbindungen prüfen

**Vor Merge**:

SAST
SCA
Code Review
Input Validation prüfen
Fehlerbehandlung prüfen
Security Tests
Regression Tests

**Dokumentieren**:

Was geändert?
Warum?
Commit
PR (Pull Request)
Testergebnis
Reviewer
Findings
Ergebnis

**3. Neue Agent-/Flow-Funktion**.

**Datenfluss aufzeichnen**:

User → Agent → Flow → Tool/API → Zielsystem

**Dann konkret prüfen**: STRIDE

Kann jemand eine andere Identity vortäuschen?
Kann Input manipuliert werden?
Kann Agent falsche Daten ins Zielsystem schreiben?
Sind Aktionen eindeutig logbar?
Können sensible Daten leaken?
Kann der Prozess blockiert werden?
Kann der Agent mehr Rechte bekommen als vorgesehen?
Welche Tools braucht er wirklich?
Welche Tools braucht er ausdrücklich nicht?
Welche Felder darf Agent schreiben?
Welche Felder nur ein Mensch?
Kann Human Approval umgangen werden?
Prompt Injection testen
Tool Failure testen
falsche/fehlende Inputs testen
Rollback möglich?

**Dokumentieren**:

Agent-/Flow-Version
Datenfluss
neue Funktion
neue Tools
neue Permissions
erkannte Threats
Controls
Testfälle
Ergebnisse
Reviewer
Approval
Deployment

**4. API / Integration bauen**

**Prüfen**:

eigene Service Identity? bsp: Copilot greift auf Jira mit copilot-jira-service. Vorteil: klare Rechte, bessere Logs.
Auth funktioniert?
minimale Berechtigungen?
welche Endpoints?
welche Methoden: GET/POST/PATCH...?
welche Felder dürfen geschrieben werden?
Felder whitelisten
Request Schema prüfen
Response Schema prüfen
400, 401, 403, 429 (Too Many Requests) behandeln
5xx behandeln
Retry/Backoff bsp: Exponential Backoff: Versuch 1 => Warte 1 sec, 2 => 2 sec, 3 => 4 sec, 4 => 8 sec.
Timeout
keine Secrets in Logs
API-Aktionen loggen

**Testen**:

korrekter Request
fehlendes Pflichtfeld
falscher Datentyp
keine Auth
keine Permission
API down
doppelte Anfrage
falsches Mapping

**Dokumentieren**:

Systeme
Endpoint
Identity
Permissions
Datenfluss
Request/Response-Beispiele
Testfälle
Fehlerfälle
Version
Owner

**5. PROD Change**

**Vorher**:

Change Ticket anlegen
Was wird geändert?
Warum?
Welche Systeme betroffen?
Security Impact?
Downtime möglich? Bsp: Durch Jira-Integration wird das System 10 Minuten nicht erreichbar.
Daten betroffen?
Permissions verändert?
Testplan vorhanden?
Rollback vorhanden?

**Dann**:

DEV → TEST → Review → Approval → PROD → Verification

**Nach Deployment**:

richtige Version?
Funktioniert Feature?
Integrationen okay?
Logs okay?
Security Controls weiterhin aktiv?
unerwartete Fehler?

**Dokumentieren**:

Change
Grund
Risk/Impact
Version
Tests
Approval
Deployment-Zeit
Deploying Person
Verification
Rollback Plan

**6. Bug / falsches Verhalten**

**Immer**:

Expected → Actual

**Dann**:

reproduzierbar?
welche Version?
welche Environment?
seit wann?
welche Inputs?
welcher Schritt ist erstmals falsch?

**Bei Integration**:

Input → Flow → Mapping → API Request → API Response → Zielsystem

**Dann**:

Root Cause
Fix
Retest
Regression Test

**Dokumentieren**:

Expected
Actual
Repro Steps
Version
Environment
Logs
Impact
Root Cause
Fix
Retest

**7. Security Finding / Vulnerability**

**Prüfen**:

welches Asset?
was ist die Schwachstelle?
wie kann sie ausgenutzt werden?
braucht Angreifer Auth?
braucht er besondere Rechte?
remote oder lokal?
was ist der Impact?
welche Systeme/Versionen betroffen?
welche Controls existieren schon?
wie kann man beheben?
welches Restrisiko bleibt?

**Dokumentieren**:

Finding
Asset
Vulnerability
Exploit Scenario
Impact
Severity/Risk
Evidence
betroffene Versionen
Owner
Fix
Deadline
Retest
Residual Risk

**8. Incident**

**Sofort**:

Was passiert gerade?
Noch aktiv?
Welche Systeme?
Welche Daten?
Welche Accounts?
seit wann?
Schaden begrenzen

**Dann**:

Contain → Scope → Root Cause → Fix → Recover

**Danach**:

welche Daten betroffen?
welche Accounts rotieren?
welche Logs sichern?
welche Systeme prüfen?
Wiederholung verhindern

**Dokumentieren**:

Timeline
Detection
Impact
Scope
Evidence
Containment: Einen laufenden Schaden begrenzen.
Root Cause
Recovery
Verantwortliche
Preventive Actions

## 9. Jira / Projektsteuerung

**Bei jedem wichtigen Ticket**:

klare Summary
Assignee
Priority
Status
Ziel
nächste Aktion
Dependency
Blocker
Due Date
relevante Links/Evidence

**Bei Blocked immer**:

Blocked by – warum – Owner – Impact – erwartete Lösung

**Bei Security Ticket**:

Asset – Finding – Risk – Evidence – Recommendation – Decision

**10. Human Approval**

**Vor Approval prüfen**:

richtige Person?
hat sie Autorität?
Evidence vollständig?
Risk bekannt?
Residual Risk bekannt?
Conditions klar?
Agent kann Entscheidung technisch nicht selbst setzen?

**Dokumentieren**:

wer
was entschieden
wann
warum
Evidence
Conditions
Residual Risk
Die eigentliche Kurzroutine

**Bei fast jeder Aufgabe**:

Was ist betroffen? → Was kann schiefgehen? → Welche Kontrolle verhindert es? → Wie teste ich das? → Was muss dokumentiert werden? → Wer entscheidet?

**11. Sandbox-Lösung evaluieren / auswählen**

**Prüfen**:

Welche Dateitypen/Artefakte können analysiert werden?
echte Isolation?
Netzwerkverkehr sichtbar/kontrollierbar?
Prozesse/File-System/Registry beobachtbar?
API vorhanden?
automatisierbar?
Reports maschinenlesbar?
Retention der Samples? Samples = Dateien/Artefakte, die zur Analyse hochgeladen werden.
wo werden Samples verarbeitet?
SSO/RBAC?
Logging/Audit Trail?
SOC/SIEM-Integration?
Kosten/API-Limits?
False Positives / False Negatives?
Sandbox-Evasion berücksichtigen?

**Testen**:

bekannte saubere Datei
bekannte verdächtige Datei
Netzwerkaktivität
API-Aufruf
Fehlerfall
Reporting

**Dokumentieren**:

Kandidaten
Bewertungskriterien
Testergebnisse
Security-/Privacy-Risiken
Integrationsaufwand
Kosten
Empfehlung
Entscheidung + Begründung

**12. ISMS / Policy / Prozess prüfen oder ändern**

**Prüfen**:

welche Policy/Procedure gilt?
welcher Scope?
welche Rollen verantwortlich?
welche Anforderungen müssen erfüllt werden?
relevante Risiken?
Controls vorhanden?
Evidence vorhanden?
Prozess noch aktuell?
Änderung notwendig?
SoA betroffen?

**Bei Änderung**:

alte Version
neue Version
warum geändert?
welche Prozesse/Systeme betroffen?
Reviewer
Approval
Inkrafttreten

**Dokumentieren**:

Requirement
Policy-/Control-Bezug
Finding/Gap
Risk
Änderung
Owner
Approval
Version
Evidence

## Wichtigste Begriffe:

**Dokumentiere**: WHAT – WHY – WHO – WHEN – VERSION – EVIDENCE – RESULT – RISK
**Asset** = schützenswertes Objekt. **interner Source Code**.
**Threat** = mögliche Schadensursache. **gestohlene Credentials verwendet**.
**Vulnerability** = Schwachstelle. **fehlendes MFA.**
**Risk** = mögliches Schadensszenario + Auswirkung/Wahrscheinlichkeit. **Zugriff auf interne Source Code.**
**Control**: Maßnahme zur Steuerung eines Risikos. MFA
**Residual Risk**: Mitigated Risk. Low
**Rollback**: Wenn die Änderung Probleme verursacht; auf einen vorherigen sicheren und funktionierenden Zustand zurückkehren.
**Change Approval**: Darf eine Systemänderung deployed werden? z.B.:Darf Agent v2.4 nach PROD?
**Security Approval**: „Residual Risk ist für diesen Use Case autorisiert akzeptabel".
**Data Retention**: Wie lange Daten gespeichert bzw. aufbewahrt werden.
**Debuggen**: Expected vs. Actual → Reproduzierbar? → Input korrekt? → Transformation korrekt? → API Request korrekt? → API Response korrekt? → Zielsystem korrekt? → Root Cause → Fix → Retest → Regression Test.
**SBOM** (Software Bill of Materials) = Liste aller Softwarekomponenten, Abhängigkeiten und Metadaten, die mit einer Anwendung verknüpft sind.
**Vendor/Processor** = euer direkter Dienstleister.
**Subprocessor** = Dienstleister des Dienstleisters, der relevante Daten mitverarbeitet.
**Environment Variable** = umgebungsspezifischer Wert.
**Connection Reference** = umgebungsspezifische Verbindung/Identity referenzieren.
**Agency** = Wie viel eigenständige Handlungsfähigkeit ein Agent in seiner Umgebung besitzt. Excessive Agency = Zu viele.

## Aus ChatGPT-Tests gelernt:

- **ISMS zu technisch gedacht:** Nicht nur Assets, Vulnerabilities und Risiken. Ein ISMS umfasst auch Policies, Rollen, Prozesse, Controls, Audits, Monitoring und kontinuierliche Verbesserung.
- **Separation of Duties:** Ziel: kritische Verantwortung auf mehrere Rollen verteilen.
- **Audit Trail:** Nicht mit Audit verwechseln. Audit = Prüfung;
  Audit Trail = chronologische Beweisspur.
- **Change Management zu eng gesehen:** Nicht nur Traceability. Änderungen sollen bewertet, entwickelt, getestet, reviewed, genehmigt, deployed, verifiziert und dokumentiert werden.
- **Agent Testing:** Teste **Systemverhalten, Berechtigungen, Regressionen, Fehlerfälle und Missbrauchsmöglichkeiten**.
- **Emergency Change:** braucht dokumentierte Bewertung, Test, Freigabe und Nachbereitung.
- **401 vs. 403:**
  `401 = Authentication`
  `403 = Authorization`
- **400:** Eher Payload/Schema/Request prüfen.
- **200/201:** HTTP-Erfolg ≠ richtige Business-Daten.
- **Debugging nicht voreilig als Angriff interpretieren:** Erst Evidence und Mapping/Input/Transformation/API/Jira-Schritte prüfen.
- **Third-Party Risks ≠ Control-Schwächen:** Risiken sind z. B. Data Breach, Ausfall, Supply-Chain-Kompromittierung, unautorisierter Zugriff.
- **EU Hosting ≠ nur EU-Verarbeitung:** Support, Logs, Backups, Subprocessors können andere Regionen betreffen.
- **Assets breiter betrachten:** Source Code, Credentials, API Keys, Developer Environment, Repository Integrity usw.
- **DEV/TEST/PROD:** Umgebungen für kontrollierte Änderungen, nicht Aufgabenbereiche des Agents.
- **Logging:** Nicht ungefiltert sensible Daten/Secrets loggen. Logs können selbst zum Risiko werden.
- **Branch ≠ Environment:** Branch = Versionszweig; Environment = Ort, an dem die Anwendung läuft.

- **Evidence:** Nicht nur Test + Approval; auch Commit, PR, Change Ticket, Deployment Log, Version und Review können Evidence sein.
- **Secrets in Git:** Löschen. Credential revoke/rotate, History & mögliche Nutzung prüfen.
- **Produktionsfehler:** Erst `Impact → Containment → Scope → Rollback/Fix`, nicht sofort entwickeln.
- **ALM-Prozess absichern:** PRs, Peer Review, Tests, Branch Protection, Change Approval, Releases, Rollback und Monitoring.

**ITIL**:
**Service Desk** = Kontakt zum Benutzer
**Incident** = Störung schnell beheben
**Problem** = Ursache finden
**Change** = Änderungen kontrollieren
**Datacenter** = physische Infrastruktur
**Server/Network/Application** = technische Systeme betreiben
**Security** = Systeme schützen
**Monitoring** = Systeme überwachen
**CMDB** = dokumentieren, was wo läuft → viel Datenpflege, Datenqualität, Excel-Import/-Export, Abgleich und Reporting.

**Outlook** = E-Mail-, Kalender- und Organisationsprogramm von Microsoft.